RELATÓRIO TÉCNICO: LABORATÓRIO DE CONJUNTOS EM JAVA

Equipe: Davi Guedes e Guilherme Carducci

Curso: Desenvolvimento de Software Multiplataforma / Matemática para Computação

Disciplina: Teoria dos Conjuntos / Matemática Discreta

1. Aplicação da Teoria dos Conjuntos no Código

A Teoria dos Conjuntos e a Álgebra Booleana foram a base estrutural de toda a arquitetura de software desenvolvida:

- **Estrutura Algorítmica Pura (Parte 1):** Para cumprir a exigência de não usar métodos prontos de conjuntos da linguagem, a classe genérica `Conjunto<T>` modelou a coleção de elementos sobre listas lineares, implementando os axiomas clássicos de conjuntos:
  - **Invariante de Não-Repetição:** Garantido na inserção através do método de pertinência ($x \notin A$), assegurando a unicidade de cada elemento ($\forall x,\ x$ é $único$).
  - **Operações Fundamentais:** União ($A \cup B$) como junção condicional de elementos; Intersecção ($A \cap B$) como filtragem por pertinência mútua; Diferença ($A - B$) via exclusão lógica de elementos de B; e Diferença Simétrica ($A \Delta B$) derivada formalmente de $(A - B) \cup (B - A)$.
  - **Inclusão:** Implementação das relações de subconjunto ($A \subseteq B$) mediante verificação de que $\forall x \in A \Longrightarrow x \in B$, e subconjunto próprio ($A \subset B$) agregando o critério estrito de cardinalidade $|A| < |B|$.
- **Categorização dos Conjuntos Numéricos (Parte 2):** A hierarquia de continência $N \subset Z \subset Q \subset R$ e $I = R - Q$ foi implementada no `ClassificadorNumerico`. A convenção $0 \in N$ foi adotada formalmente (norma ISO 80000-2). A dedução da fração geratriz para dízimas compostas baseou-se na álgebra de progressões geométricas infinitas sintetizada na regra do MDC com números 9 e 0.
- **Mini-projeto Prático - Similaridade de Jaccard (Parte 3):** A aplicação avançada adotou o Índice de Jaccard, que calcula a proximidade métrica entre dois conjuntos finitos de interesses:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

O algoritmo encontra o usuário de maior afinidade e aplica a diferença relativa de conjuntos ($B - A$) para gerar as recomendações — recomendando ao usuário alvo os itens que pertencem ao perfil semelhante, mas que o alvo ainda não possui.

2. Operação/Conceito Mais Desafiador e Justificativa

O desafio técnico e conceitual mais complexo do laboratório foi o cálculo automatizado da fração geratriz de dízimas periódicas compostas e o tratamento de suas singularidades.

Justificativa: Diferente das dízimas simples (como $0,333\ldots \rightarrow 3/9$), uma dízima composta (como $1,2(5)$) exige a decomposição léxica de três componentes independentes: parte inteira (1), anteperíodo (2) e período (5). A implementação exigiu:

1. **Modelagem aritmética precisa** para subtrair a parte não periódica do número expandido sem sofrer imprecisão binária de tipos de ponto flutuante (`float`/`double`):

$$Numerador = \left(Parte\ Inteira \cdot 10^{k} + Anteperíodo\right) \cdot 10^{p} + Período - \left(Parte\ Inteira \cdot 10^{k} + Anteperíodo\right)$$

2. **Montagem do denominador padronizado** com tantos noves quantos forem os dígitos do período e tantos zeros quantos forem os dígitos do anteperíodo (90).
3. **Redução da fração** à sua forma irredutível via Algoritmo de Euclides para cálculo do MDC (Máximo Divisor Comum).
4. **Tratamento do caso de borda do período 9:** Dízimas como $0,(9)$ ou $1,(9)$ geram frações que convergem analiticamente para números inteiros ($9/9 = 1$ e $18/9 = 2$). O código precisou reconhecer quando o denominador simplificado converge para 1, reclassificando o número como inteiro e natural ($\in N, Z$), rompendo a heurística ingênua de que toda dízima é estritamente não inteira.

3. Conclusão

O projeto integrou com êxito os fundamentos matemáticos à engenharia de software em Java. A verificação cruzada contra `java.util.Set` demonstrou 100% de conformidade em todos os testes de borda, validando a robustez dos algoritmos desenvolvidos do zero.